Documento de Arquitetura de Software

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
[20/04/2026]	[]	[Maria Clara]	[Elaborar e adicionar os dois primeiros tópicos descrevendo a arquitetura]	[1.0]
[20/04/2026]	[]	[Luiz Guilherme]	[Elaboração do Documento]	[1.1]
[20/04/2026]	[]	[Maria Fernanda]	[Adicionar os últimos tópicos do Documento]	[1.2]

Sumário

1. Introdução 3
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, acrônimos e abreviações 3
1.4 Referências 4
2. Descrição geral da arquitetura 4
3. Estilo arquitetural adotado 5
4. Principais Componentes e suas Responsabilidades 5
5. Diagrama de Arquitetura 6
6. Tecnologias Previstas 6
6.1 Linguagem de Programação: 7
6.2 Bibliotecas padrão: 7
6.3 Bibliotecas para processamento de arquivos: 7
6.4 Frameworks: 7
6.5 Banco de Dados: 7
6.6 Ferramentas de desenvolvimento: 7
7. Justificativa das Decisões Arquiteturais 7

1. Introdução

1.1 Finalidade

Este documento tem como finalidade descrever a arquitetura do sistema de scanner de arquivos desenvolvido, detalhando sua estrutura, organização e principais componentes. O objetivo é fornecer uma visão clara do funcionamento do sistema, permitindo o entendimento por parte da equipe técnica e demais interessados. Além disso, o documento apresenta as decisões arquiteturais adotadas, bem como as tecnologias utilizadas no desenvolvimento, servindo como base para manutenção e evolução do sistema.

1.2 Escopo

A aplicação consiste em uma aplicação de linhas de comando voltada para análise e varredura de arquivos locais, incluindo PDFs, planilhas e arquivos compactados.

A aplicação permite ao usuário informar diretórios ou arquivos específicos para processamento, realizando a leitura, extração e análise dos dados com base em regras previamente definidas. Como resultado, o sistema gera informações estruturadas que podem ser exibidas no terminal ou exportadas em formato de relatório.

O escopo deste documento abrange a descrição da arquitetura do sistema, incluindo seus componentes, organização em camadas, tecnologias utilizadas e justificativas das decisões arquiteturais.

1.3 Definições, acrônimos e abreviações

CLI (Command Line Interface): Interface de linha de comando que permite a interação com o sistema por meio de comandos digitados no terminal.

PDF (Portable Document Format): Formato de arquivo amplamente utilizado para documentos digitais.

ZIP: Formato de compressão de arquivos que permite agrupar múltiplos arquivos em um único arquivo compactado.

Pipeline de processamento: Sequência de etapas pelas quais os dados passam, desde a entrada até a geração de resultados.

Módulo: Parte do sistema responsável por uma funcionalidade específica.

Processamento assíncrono: Execução de tarefas em segundo plano, sem bloquear o fluxo principal da aplicação.

1.4 Referências

- Bibliotecas subprocess, os, re, math, sys
- Materiais da disciplina de Engenharia de Software
- Pressman, R. S. – Engenharia de Software
- Sommerville, I. – Engenharia de Software
- Documentação oficial da linguagem Python

2. Descrição geral da arquitetura

O sistema consiste em uma aplicação de linha de comando (CLI) desenvolvida para realizar a varredura e análise de arquivos locais, com foco na identificação de padrões, extração de informações relevantes e detecção de possíveis riscos de segurança.

A arquitetura é estruturada de forma modular, organizada em um fluxo de processamento composto por etapas bem definidas: entrada de dados, processamento, análise e geração de saída. O usuário interage com o sistema por meio de comandos no terminal, informando os arquivos ou diretórios a serem analisados.

Durante a execução, o sistema realiza a leitura de diferentes tipos de arquivos, como PDFs, planilhas e arquivos compactados, efetuando a extração e normalização dos dados. Em seguida, aplica regras de análise para identificar padrões e inconsistências. Os resultados são então apresentados ao usuário no terminal ou exportados em formato estruturado.

Essa organização permite maior flexibilidade, facilitando a manutenção, evolução e reutilização dos componentes do sistema.

2.1 Visão Geral dos Módulos

2.1.1 Módulo de Ingestão e Extração (Git Ingestion Module):

- Responsável por se conectar via API REST / GraphQL do GitHub/GitLab com permissões estritas de leitura (*read-only*).
- Coleta metadados de commits, mensagens, *timestamps*, alteração de linhas (*churn*) e identificadores de autores.

2.1.2 Motor de Análise e Métricas (Metrics & Ownership Engine):

- Calcula o índice de concentração de conhecimento por repositório, diretório e arquivo.
- Mapeia *hotspots* correlacionando volume de linhas alteradas e frequência de commits.

2.1.3 Motor Preditivo de Simulação (Risk Simulation Engine):

- Processa algoritmos de simulação baseados na remoção hipotética de nós (desenvolvedores) do grafo de conhecimento.
- Modela e classifica os níveis de risco (Baixo, Médio, Crítico).

2.1.4 Camada de Apresentação (Dashboard / CLI):

- Exibe visualmente o mapa de calor de arquivos, gráficos interativos do ecossistema do projeto e sugestões de mitigação.

3. Estilo arquitetural adotado

O sistema adota o estilo arquitetural em camadas, aliado ao modelo de aplicação baseada em linha de comando (CLI).

A arquitetura em camadas permite a separação de responsabilidades, organizando o sistema em diferentes níveis funcionais. A camada de interface é responsável pela interação com o usuário via terminal, recebendo comandos e parâmetros de execução. A camada de aplicação coordena o fluxo de execução do sistema, controlando a sequência das operações. A camada de processamento realiza a leitura e tratamento dos arquivos, enquanto a camada de análise aplica regras e validações sobre os dados extraídos. Por fim, a camada de saída é responsável pela apresentação dos resultados.

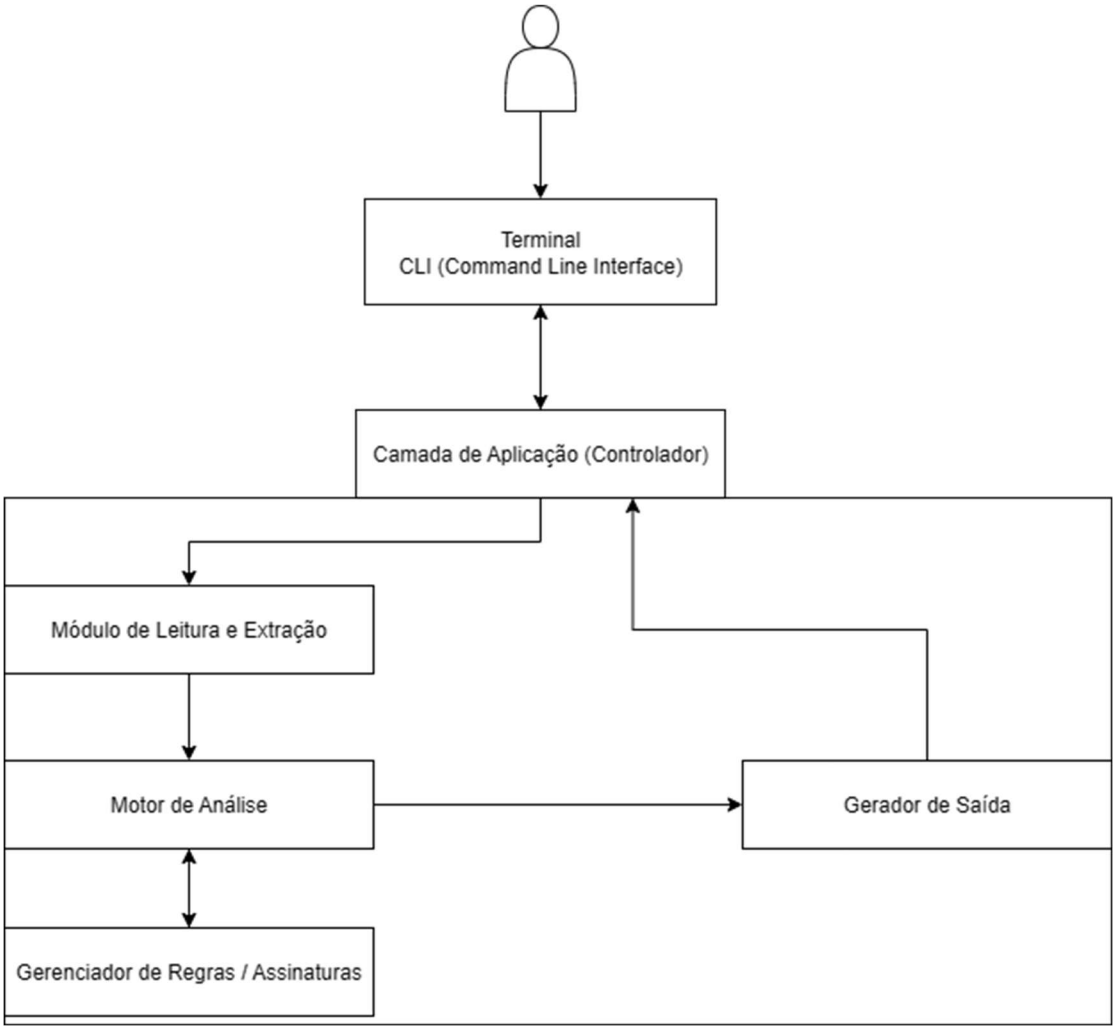
Esse modelo favorece a modularidade, facilitando a manutenção e permitindo a evolução do sistema, como a possível integração futura com outras interfaces, como aplicações web ou APIs.

4. Principais Componentes e suas Responsabilidades

- Terminal / CLI (Command Line Interface): Representa a camada de interface, sendo a porta de entrada para os comandos do usuário.

- Camada de Aplicação (Controlador): Coordena o fluxo de execução do sistema, controlando a sequência das operações e a comunicação entre a interface e os módulos internos.
- Módulo de Leitura e Extração: Responsável pela leitura e tratamento de arquivos (PDFs, planilhas, compactados) para extração de dados brutos.
- Motor de Análise: Aplica regras e validações sobre os dados extraídos para identificar padrões e possíveis riscos de segurança.
- Gerenciador de Regras / Assinaturas: Fornece as definições de padrões e regras previamente estabelecidas que orientam a análise.
- Gerador de Saída: Responsável pela apresentação final dos resultados, formatando as informações para o terminal ou relatórios exportados.

5. Diagrama de Arquitetura



6. Tecnologias Previstas

O sistema será desenvolvido utilizando a linguagem ***Python***, escolhida por sua simplicidade, legibilidade e ampla disponibilidade de bibliotecas para manipulação de arquivos e processamento de dados.

As principais tecnologias e bibliotecas previstas incluem:

6.1 Linguagem de Programação:

- Python 3.x

6.2 Bibliotecas padrão:

- “os”: manipulação de arquivos e diretórios
- “re”: processamento de expressões regulares
- “sys”: interação com parâmetros de execução
- “subprocess”: execução de comandos externos
- “math”: operações matemáticas auxiliares

6.3 Bibliotecas para processamento de arquivos:

- “PyPDF2” ou “pdfplumber”: leitura e extração de dados de arquivos PDF
- “pandas”: manipulação de dados estruturados (planilhas)
- “openpyxl”: leitura de arquivos Excel
- “zipfile”: manipulação de arquivos compactados

6.4 Frameworks:

Não há uso de frameworks robustos, pois a aplicação é baseada em linha de comando (CLI), sendo construída de forma modular com bibliotecas nativas e externas.

6.5 Banco de Dados:

- Não há utilização de banco de dados nesta versão do sistema.
- Os dados são processados em memória e os resultados podem ser exportados em arquivos (como “.txt”, “.csv” ou “.json”).

6.6 Ferramentas de desenvolvimento:

- Git (controle de versão)
- GitHub ou GitLab (repositório remoto)
- VS Code ou similar (ambiente de desenvolvimento)

7. Justificativa das Decisões Arquiteturais

As decisões arquiteturais adotadas no desenvolvimento do sistema foram baseadas em critérios de simplicidade, modularidade, desempenho e facilidade de manutenção.

A escolha da arquitetura em camadas foi motivada pela necessidade de separar responsabilidades dentro do sistema, permitindo que cada parte tenha uma função bem definida. Isso facilita a manutenção, testes e evolução do sistema, além de reduzir o acoplamento entre os componentes.

A utilização de uma aplicação baseada em linha de comando (CLI) foi definida por ser uma solução mais leve e adequada ao propósito do sistema, que é a análise de arquivos locais. Esse modelo elimina a necessidade de interfaces gráficas complexas e reduz o consumo de recursos.

A escolha da linguagem Python se deve à sua alta produtividade, grande quantidade de bibliotecas disponíveis e facilidade na manipulação de arquivos e dados. Isso permite um desenvolvimento mais rápido e eficiente.

A adoção de uma arquitetura modular com pipeline de processamento permite que o sistema execute etapas bem definidas (entrada, processamento, análise e saída), facilitando a reutilização de componentes e a inclusão de novas funcionalidades no futuro.

A decisão de não utilizar banco de dados nesta versão foi tomada devido à natureza do sistema, que realiza processamento pontual de arquivos. Isso reduz a complexidade da aplicação e melhora o desempenho, uma vez que não há necessidade de persistência contínua dos dados.

Por fim, o uso de bibliotecas específicas para leitura de arquivos (PDF, Excel, ZIP) garante maior eficiência e confiabilidade na extração de dados, evitando a necessidade de implementação manual dessas funcionalidades.